Results Aggregation Script

Oct 21, 2026 · @Fasya

What it is for

`sentiment/aggregate_results.py` turns the 9 training runs (3 models × seeds 42, 123, 999) into one mean ± std results table for the paper. This is Member 3's Milestone 3 task. The script is written and tested, but it is not pushed to GitHub yet, so the team can review it here first.

We train each model 3 times with different random seeds because one run can be lucky or unlucky. The **mean** is the typical score. The **std** (standard deviation) shows how much the score moves between runs: a small std means the result is stable.

Real results: Method A and B, 3 seeds each

On the test set, Method A (XLM-RoBERTa) beat Method B (IndoBERT + class weights): Macro-F1 was 0.899 ± 0.015 vs 0.865 ± 0.020. That means our hypothesis that Method B would score best is **not supported** so far. On the validation set the two are almost tied (0.917 vs 0.913). All 6 runs were trained on 21–22 Oct 2026 on Fasya's laptop (NVIDIA RTX 3050, 4 GB), using the same settings: 5 epochs, batch 4 × 4 accumulation, learning rate 2e-5. The Baseline isn't included yet because it has no training script.

Validation set (mean ± std over seeds 42, 123, 999)

Model	Accuracy	Macro-F1	Weighted-F1
Method A: XLM-RoBERTa	0.9389 ± 0.0021	0.9167 ± 0.0016	0.9388 ± 0.0019
Method B: IndoBERT + class weights	0.9386 ± 0.0056	0.9133 ± 0.0035	0.9383 ± 0.0053

Test set (mean ± std over seeds 42, 123, 999)

Model	Accuracy	Macro-F1	Weighted-F1
Method A: XLM-RoBERTa	**0.9207 ± 0.0081**	**0.8988 ± 0.0154**	**0.9181 ± 0.0095**
Method B: IndoBERT + class weights	0.8960 ± 0.0122	0.8647 ± 0.0198	0.8915 ± 0.0139

Each run (test set)

Model	Seed	Best epoch	Accuracy	Macro-F1	Weighted-F1	Training time (min)
Method A	42	3	0.9280	0.9102	0.9258	119.6
Method A	123	5	0.9120	0.8812	0.9075	115.4
Method A	999	4	0.9220	0.9048	0.9209	113.6
Method B	42	4	0.8900	0.8520	0.8837	22.6
Method B	123	3	0.9100	0.8876	0.9075	22.3
Method B	999	4	0.8880	0.8546	0.8831	22.5

Size and cost: Method A has 278.0 million parameters, and Method B has 124.4 million. Method A took about 116 minutes per run and Method B about 22 minutes. Method A's time is inflated, though: it didn't fit in the 4 GB of GPU memory, so part of it spilled into normal RAM. Don't report this as a fair speed comparison without that note.

Things to look into for the paper:

- Method B's validation scores were as good as Method A's, but it dropped more on test (Macro-F1 0.913 → 0.865). Its test loss is also much higher (about 1.05 vs 0.39). This may mean the class weights make it over-confident when it is wrong.
- The test set has many more Neutral sentences (17.6%) than the training set (10.4%). A per-class F1 breakdown and confusion matrix (Milestone 4) will show whether Neutral is where Method B loses.
- The raw per-seed numbers are in `all_runs.csv`, and the mean ± std table is in `results_table.csv`.

How it works

Run it from the repo's top folder after training:

```
python -m sentiment.aggregate_results
```

1. **Find the run files.** Each training run saves a `summary.json` at `outputs/<model>/seed_<seed>/summary.json` (for example `outputs/method_a/seed_42/summary.json`). `train_transformers.py` already writes this file, holding the final Val and Test scores of the best checkpoint.
2. **Check for missing runs.** The script ticks off the 9 expected runs and prints a warning listing any that are missing. It still builds the table from the runs it has.
3. **Compute mean ± std.** For each model it averages Accuracy, Macro-F1 and Weighted-F1 on Val and Test, plus training time. The std is the *sample* std (it divides by n − 1), the same as the existing `aggregate_runs` in `evaluation.py`.
4. **Save and print.** It writes `outputs/all_runs.csv` (one row per seed) and `outputs/results_table.csv` (one row per model, with raw numbers for plots and "mean +/- std" text for the paper). It also prints the tables below.

The formulas, for one model with n seeds and scores $x_1 \dots x_n$:

```
\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n} (x_i - \bar{x})^2}
```

The script only needs plain Python and doesn't load PyTorch, so it runs on any laptop.

Example output (made-up numbers)

These numbers are fake. No real training runs exist yet, so I tested the script on randomly generated `summary.json` files. Baseline seed 999 was left out on purpose to test the warning. The real table will look the same, filled with real scores.

What the script printed:

```
Found 8 run(s) under outputs
WARNING: 1 expected run(s) missing: baseline/seed_999
```

Val set (mean +/- std over seeds)

Model	Seeds	Accuracy	Macro-F1	Weighted-F1
baseline	2	0.7438 +/- 0.0166	0.6822 +/- 0.0018	0.7436 +/- 0.0037
method_a	3	0.8496 +/- 0.0155	0.7826 +/- 0.0014	0.8507 +/- 0.0126
method_b	3	0.8996 +/- 0.0155	0.8326 +/- 0.0014	0.9007 +/- 0.0126

Test set (mean +/- std over seeds)

Model	Seeds	Accuracy	Macro-F1	Weighted-F1
baseline	2	0.7366 +/- 0.0033	0.7128 +/- 0.0047	0.7443 +/- 0.0181
method_a	3	0.8421 +/- 0.0097	0.8084 +/- 0.0083	0.8413 +/- 0.0138
method_b	3	0.8921 +/- 0.0097	0.8584 +/- 0.0083	0.8913 +/- 0.0138

Cost (the fake files all used the same parameter count; real models will differ)

Model	Parameters	Training seconds
baseline	278,045,955	1082.5 +/- 57.3
method_a	278,045,955	1388.0 +/- 530.7
method_b	278,045,955	1388.0 +/- 530.7

Open questions for the team

- [ ] **Member 1:** The Baseline has no training script yet. Can it save results to `outputs/baseline/seed_<seed>/summary.json`, with the same `metrics` → `val` / `test` → `accuracy`, `macro_f1`, `weighted_f1` fields as `train_transformers.py`?
- [ ] **Member 2:** Training on Colab writes `outputs/` there. The folder is git-ignored, so the 9 run folders need to be copied (for example via Google Drive) into one `outputs/` folder before running this script.
- [ ] **Everyone:** Should the script also print the table in LaTeX format, ready to paste into Overleaf?

Full code

Here is the whole file, `sentiment/aggregate_results.py`. The comment on each line explains what it does. To suggest a change, leave a comment here.

```
"""Gather every run's summary.json and report mean +/- std per model.

Expected layout (written by train_transformers.py):
    outputs/<model>/seed_<seed>/summary.json
for example outputs/method_a/seed_42/summary.json. Each summary.json holds
the final Val and Test scores of the checkpoint picked by validation Macro-
F1.

Usage:
    python -m sentiment.aggregate_results
    python -m sentiment.aggregate_results --output-dir outputs
"""

import argparse                          # reads options typed on the command
line
import csv                               # writes the result tables as .csv
files
import json                              # reads each run's summary.json file
from pathlib import Path                 # handles folder and file paths
safely
from statistics import mean, stdev       # average and sample standard
deviation

ROOT = Path(__file__).resolve().parent.parent   # the repository's top folder

# The plan: 3 models x 3 seeds = 9 runs. The baseline folder name assumes the
# baseline trainer will follow the same outputs/<model>/seed_<seed>/ layout.
EXPECTED_MODELS = ("baseline", "method_a", "method_b")
EXPECTED_SEEDS = (42, 123, 999)
SPLITS = ("val", "test")                         # the two sets we report on
METRIC_KEYS = ("accuracy", "macro_f1", "weighted_f1")   # the paper's metrics


def find_runs(output_dir: Path) -> dict[str, dict[int, dict]]:
    """Load every outputs/<model>/seed_<seed>/summary.json into a nested
dict.

    The result looks like {"method_a": {42: {...summary...}, 123: {...}},
...}.
    """
    runs: dict[str, dict[int, dict]] = {}                    # start empty
    # glob finds every file matching the pattern, one folder level per "*"
    for path in sorted(output_dir.glob("*/seed_*/summary.json")):
```

```
        model = path.parent.parent.name                          # e.g.
"method_a"
        seed_text = path.parent.name.removeprefix("seed_")    # "seed_42" ->
"42"
        if not seed_text.isdigit():                              # skip odd
folders
            print(f"Skipping {path}: folder name is not seed_<number>")
            continue
        summary = json.loads(path.read_text())                # read the JSON
file
        if "metrics" not in summary:                             # must hold
scores
            raise ValueError(f"{path} has no 'metrics' section")
        runs.setdefault(model, {})[int(seed_text)] = summary  # store under
model+seed
    return runs


def report_missing(runs: dict[str, dict[int, dict]]) -> list[str]:
    """List every expected (model, seed) pair that has no summary.json
yet."""
    missing = []
    for model in EXPECTED_MODELS:                    # check each planned
model
        for seed in EXPECTED_SEEDS:                  # ...and each planned
seed
            if seed not in runs.get(model, {}):      # no file found for this
pair
                missing.append(f"{model}/seed_{seed}")
    return missing


def mean_std(values: list[float]) -> tuple[float, float]:
    """Return (mean, sample std). One value has no spread, so std is 0."""
    return mean(values), (stdev(values) if len(values) > 1 else 0.0)


def per_run_rows(runs: dict[str, dict[int, dict]]) -> list[dict]:
    """One flat row per run: model, seed, then every split/metric score."""
    rows = []
    for model, by_seed in sorted(runs.items()):          # models in A-Z
order
        for seed, summary in sorted(by_seed.items()):     # seeds low to high
```

```
            row = {"model": model, "seed": seed}
            for split in SPLITS:                          # val, then test
                for key in METRIC_KEYS:                   # accuracy,
macro_f1, ...
                    row[f"{split}_{key}"] = summary["metrics"][split][key]
            row["training_seconds"] = summary.get("training_seconds")       #
time cost
            row["parameters"] = summary.get("parameters", {}).get("total")  #
model size
            rows.append(row)
    return rows


def aggregate_rows(rows: list[dict], digits: int) -> list[dict]:
    """Collapse the per-run rows into one mean +/- std row per model."""
    columns = [f"{split}_{key}" for split in SPLITS for key in METRIC_KEYS]
    columns.append("training_seconds")
    table = []
    models = sorted({row["model"] for row in rows})               # unique
model names
    for model in models:
        model_rows = [row for row in rows if row["model"] == model]   # this
model's seeds
        summary = {"model": model, "n_seeds": len(model_rows)}
        for column in columns:
            values = [row[column] for row in model_rows if row[column] is not
None]
            if not values:                                        # nothing
recorded
                summary[column] = ""
                continue
            avg, spread = mean_std([float(value) for value in values])
            summary[f"{column}_mean"] = avg                       # raw
numbers for plots
            summary[f"{column}_std"] = spread
            # the human-readable form used in the paper's results table;
            # seconds only need 1 decimal place, scores use --digits
            places = 1 if column == "training_seconds" else digits
            summary[column] = f"{avg:.{places}f} +/- {spread:.{places}f}"
        # parameter count is the same for every seed, so report it once
        summary["parameters"] = model_rows[0]["parameters"]
        table.append(summary)
    return table
```

```
def write_csv(path: Path, rows: list[dict]) -> None:
    """Save a list of dicts as a CSV file, one dict per line."""
    fields = list(dict.fromkeys(key for row in rows for key in row))   # all
column names, in order
    with path.open("w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields)
        writer.writeheader()          # first line: the column names
        writer.writerows(rows)        # then one line per row


def print_table(table: list[dict]) -> None:
    """Print the mean +/- std table as Markdown, ready to paste or read."""
    for split in SPLITS:
        print(f"\n### {split.capitalize()} set (mean +/- std over seeds)\n")
        print("| Model | Seeds | Accuracy | Macro-F1 | Weighted-F1 |")
        print("| :--- | :---: | :---: | :---: | :---: |")
        for row in table:
            scores = " | ".join(row[f"{split}_{key}"] for key in METRIC_KEYS)
            print(f"| {row['model']} | {row['n_seeds']} | {scores} |")
    print("\n### Cost\n")
    print("| Model | Parameters | Training seconds |")
    print("| :--- | ---: | :---: |")
    for row in table:
        params = f"{row['parameters']:,}" if row["parameters"] else "?"   #
1,234,567 style
        print(f"| {row['model']} | {params} | {row['training_seconds'] or
'?'} |")


def main():
    parser = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    parser.add_argument("--output-dir", type=Path, default=ROOT / "outputs",
                        help="Folder that holds
<model>/seed_<seed>/summary.json")
    parser.add_argument("--digits", type=int, default=4, help="Decimal places
(default: 4)")
    args = parser.parse_args()

    try:
        runs = find_runs(args.output_dir)
    except (OSError, ValueError, KeyError) as exc:
```

```
        parser.exit(1, f"Aggregation failed: {exc}\n")
    if not runs:
        parser.exit(1, f"No summary.json files found under
{args.output_dir}\n")

    found = sum(len(by_seed) for by_seed in runs.values())    # how many runs
we have
    print(f"Found {found} run(s) under {args.output_dir}")
    missing = report_missing(runs)
    if missing:   # warn, but still aggregate whatever exists
        print(f"WARNING: {len(missing)} expected run(s) missing: {',
'.join(missing)}")

    try:
        rows = per_run_rows(runs)
    except KeyError as exc:   # a summary.json without val/test scores
        parser.exit(1, f"Aggregation failed: a summary.json is missing
{exc}\n")
    table = aggregate_rows(rows, args.digits)

    write_csv(args.output_dir / "all_runs.csv", rows)          # every seed,
raw scores
    write_csv(args.output_dir / "results_table.csv", table)    # mean +/- std
per model
    print_table(table)
    print(f"\nSaved {args.output_dir / 'all_runs.csv'} and {args.output_dir /
'results_table.csv'}")


if __name__ == "__main__":
    main()
```